

MODULE NAME: PYTHON BDD RESTFUL AUTOMATIONS

EXPERIMENT TITLE: LOCATE THE LEFT-HAND SIDE TEXT BOX

Problem Statement

To identify and locate the **left-hand side text box** on the Text Compare website using Selenium WebDriver and an appropriate locator strategy.

Objective

To learn how to locate a specific web element based on its position and interact with it using Selenium.

Tools, Software, and Concepts Used

- **Python 3.x**
- **Selenium (latest version)**
- **Google Chrome**
- **Selenium WebDriver**
- **VS Code / PyCharm**
- Python packages:
 - selenium
 - webdriver-manager
- Concepts: **Web Element Identification, XPath, CSS Selector, Textarea, Relative Locators**

Selenium supports standard locators including XPath, CSS selector, ID, class name, and tag name.

Source Code / Implementation

For this page, the two comparison boxes are text areas. The following code identifies the **first/left-hand text box** using `textarea[0]`.

```
from selenium import webdriver

from selenium.webdriver.chrome.service import Service

from selenium.webdriver.common.by import By

from webdriver_manager.chrome import ChromeDriverManager

import time

service = Service(ChromeDriverManager().install())

driver = webdriver.Chrome(service=service)

driver.get("https://text-compare.com/")
```

```
driver.maximize_window()

time.sleep(3)

# Locate the left-hand side text box

text_boxes = driver.find_elements(By.TAG_NAME, "textarea")

left_box = text_boxes[0]

# Click the left text box

left_box.click()

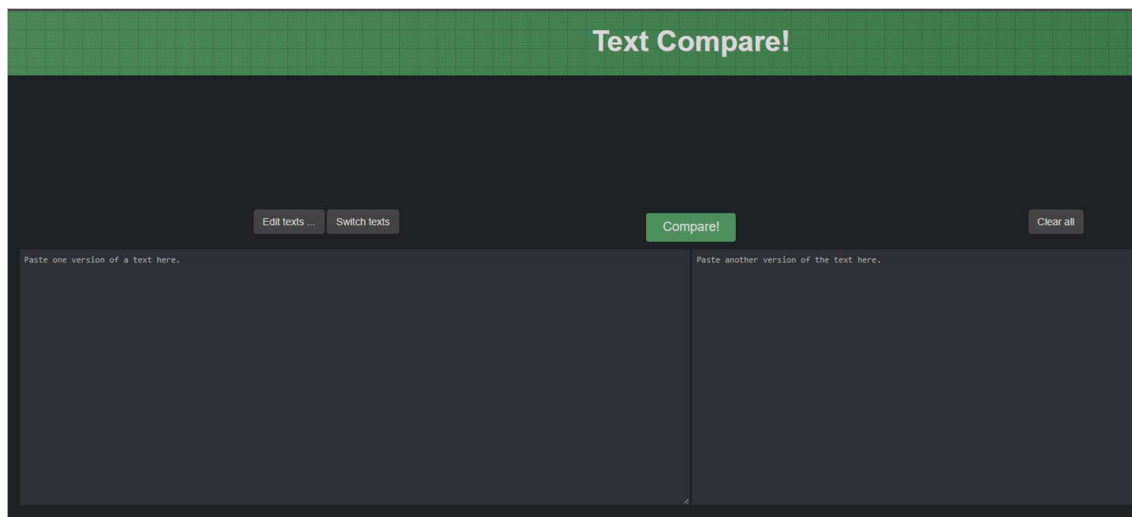
left_box.send_keys("This is the left-hand side text box.")

print("Left-hand side text box located successfully.")

time.sleep(3)

driver.quit()
```

Output with clearly labelled screenshots :



Result: The left-hand side text box was successfully located and accessed using Selenium.

Observation: `find_elements()` returns multiple matching elements, and the first element can be accessed using index 0.

Conclusion: The experiment demonstrated how Selenium can identify a specific web element among multiple similar elements and interact with it.

EXPERIMENT TITLE: DRAG AND DROP USING AUTOMATION

Problem Statement

To identify a draggable web element and a drop target on a webpage and perform a drag-and-drop operation using Selenium WebDriver.

Objective

To learn how to automate drag-and-drop operations using Selenium's **ActionChains** class.

Tools, Software, and Concepts Used

- **Python 3.x**
- **Selenium (latest version)**
- **Google Chrome**
- **Selenium WebDriver**
- **VS Code / PyCharm**
- **Python packages:**
 - `selenium`
 - `webdriver-manager`
- **Concepts: Drag and Drop, ActionChains, Web Element Identification, `drag_and_drop()`**

The given website has a **Drag and Drop** section with "**Drag me to my target**" and "**Drop here**" elements.

Source Code / Implementation :

```
from selenium import webdriver

from selenium.webdriver.chrome.service import Service

from selenium.webdriver.common.by import By

from selenium.webdriver.common.action_chains import ActionChains

from webdriver_manager.chrome import ChromeDriverManager

import time

service = Service(ChromeDriverManager().install())

driver = webdriver.Chrome(service=service)

driver.get("https://testautomationpractice.blogspot.com/")

driver.maximize_window()

time.sleep(3)

source = driver.find_element(

    By.XPATH, "//div[contains(text(),'Drag me to my target')]")

target = driver.find_element(

    By.XPATH, "//div[contains(text(),'Drop here')]")

actions = ActionChains(driver)

actions.drag_and_drop(source, target).perform()
```

```
time.sleep(3)

print("Drag and drop operation completed successfully.")

driver.quit()
```

Result: The draggable element was successfully moved and dropped onto the target area.

Observation: Selenium's `ActionChains` class can simulate mouse-based actions such as drag and drop.

Conclusion: The experiment demonstrated how to automate drag-and-drop interactions using Selenium WebDriver and `ActionChains`.

Available next action: Create a downloadable DOCX file here in this chat containing the editable prose above

EXPERIMENT TITLE: WRITE TEXT IN THE BOX

Problem Statement

To identify a text input box on a webpage and enter the required text into it automatically using Selenium WebDriver.

Objective

To learn how to locate a text box and enter text using Selenium's `send_keys()` method.

Tools, Software, and Concepts Used

- Python 3.x
- Selenium (latest version)
- Google Chrome
- Selenium WebDriver
- VS Code / PyCharm
- Python packages:
 - selenium
 - webdriver-manager
- Concepts: **Web Element Identification, Text Box, `send_keys()`, XPath, CSS Selector**

Source Code / Implementation :

```
from selenium import webdriver

from selenium.webdriver.chrome.service import Service

from selenium.webdriver.common.by import By

from webdriver_manager.chrome import ChromeDriverManager

import time
```

```
service = Service(ChromeDriverManager().install())

driver = webdriver.Chrome(service=service)

driver.get("https://testautomationpractice.blogspot.com/")

driver.maximize_window()

time.sleep(3)

name_box = driver.find_element(By.ID, "name")

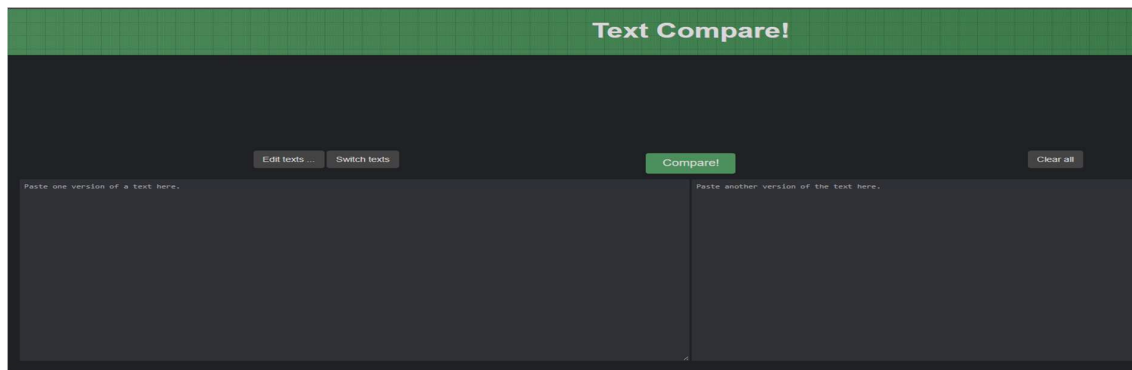
name_box.send_keys("Parthib Kumar Das")

print("Text entered successfully.")

time.sleep(3)

driver.quit()
```

Output with clearly labelled screenshots :



Result: The required text was successfully entered into the text box.

Observation: The `send_keys()` method is used to enter text into input fields.

Conclusion: The experiment demonstrated how Selenium can locate a text box and automatically enter text into it.

Available next action: Create a downloadable DOCX file here in this chat containing the editable prose above

Name: Aasish Shrestha , Enrollment No: 12023002001003 , CSE , Roll No: 33
Institute of Engineering & Management, Kolkata
Email ID: aasish.shrestha2023@iem.edu.in
aasishshrestha2005@gmail.com